SMART CAREER ADVANCEMENT PLATFORM (AI-POWERED CAREER ASSISTANT)

A project report submitted in partial fulfillment of the requirements for the award of degree of

BACHELOR OF COMPUTER APPLICATIONS
of
Dr. Manmohan Singh Bengaluru City University

By
HARISHWAR S
U18HGH24S0053

Under the Guidance of
Mrs. SOWMYA K
Assistant Professor

SESHADRIPURAM COLLEGE
27, Nagappa Street, Seshadripuram, Bengaluru-560 020 Affiliated to Dr. Manmohan Singh Bengaluru City University
ISO 9001:2015 Certified Recognized Under Section 2(f) and 12(B) of the University Grants Commission Act, 1956
AICTE Approved NAAC Grade: A++
Website: www.spmcollege.ac.in www.pg.spmcollege.ac.in

2025-26

SMART CAREER ADVANCEMENT PLATFORM (AI-POWERED CAREER ASSISTANT)

A project report submitted in partial fulfillment of the requirements for the award of degree of

BACHELOR OF COMPUTER APPLICATIONS
of
Dr. Manmohan Singh Bengaluru City University

By
HARISHWAR S
U18HGH24S0053

Under the Guidance of
Mrs. SOWMYA K
Assistant Professor

SESHADRIPURAM COLLEGE
27, Nagappa Street, Seshadripuram, Bengaluru-560 020 Affiliated to Dr. Manmohan Singh Bengaluru City University
ISO 9001:2015 Certified Recognized Under Section 2(f) and 12(B) of the University Grants Commission Act, 1956
AICTE Approved NAAC Grade: A++
Website: www.spmcollege.ac.in www.pg.spmcollege.ac.in

2025-26

SESHADRIPURAM COLLEGE
27, Nagappa Street, Seshadripuram, Bengaluru-560 020 Affiliated to Dr. Manmohan Singh Bengaluru City University
ISO 9001:2015 Certified Recognized Under Section 2(f) and 12(B) of the University Grants Commission Act, 1956
AICTE Approved NAAC Grade: A++
Website: www.spmcollege.ac.in www.pg.spmcollege.ac.in

CERTIFICATE

This is to Certify that **HARISHWAR S** bearing USN: **U18HGH24S0053** has completed his final semester project work entitled **"CAREER ADVANCEMENT PLATFORM (AI-POWERED CAREER ASSISTANT)"** as a partial fulfillment for the award of Bachelor of Computer Applications degree, during the academic year 2025-26 under my supervision.

Signature of Internal Guide
Mrs. Sowmya K
Assistant Professor

Signature of HOD
Prof. Rajeswari V
Head of the Department

External Examiner: .....

Internal Examiner: .....

DECLARATION

I, **HARISHWAR S**, student of 6th semester BCA, **SESHADRIPURAM COLLEGE**, bearing USN: **U18HGH24S0053** hereby declare that the project entitled **CAREER ADVANCEMENT PLATFORM (AI-POWERED CAREER ASSISTANT)** has been carried out by me under the supervision of Internal Guide **Mrs. SOWMYA K**, Assistant Professor and submitted in partial fulfillment of the requirements for the award of the Degree of Bachelor of Computer Applications by the **Dr. Manmohan Singh Bengaluru City University** during the academic year 2025-26. This report has not been submitted to any other Organization/University for any award of degree or certificate.

Name: HARISHWAR S

Signature: .....

ACKNOWLEDGEMENT

I take this opportunity to first express my heartfelt gratitude to God for providing me with the strength, wisdom, and perseverance to successfully complete my project and this report.

I would also like to sincerely thank my parents for their constant support, encouragement, and belief in me throughout this journey. Their guidance and motivation have been a strong foundation for my achievements.

First and foremost, I am extremely thankful to **Maj Dr. Anandappa I**, Principal, Seshadripuram College, for providing me with the platform and encouragement to take up this project as part of my academic curriculum.

I extend my heartfelt thanks to **Prof. Rajeswari V**, Head of the Department of Computer Science, for her constant support, motivation, and for facilitating a smooth academic environment.

I would also like to express my deep appreciation to my internal project guide **Mrs. Sowmya K**, Assistant Professor, for her invaluable guidance, continuous feedback, and encouragement throughout the course of the project. Her technical insights and support played a crucial role in successfully completing this experience.

Lastly, I thank all faculty members of the Department of Computer Science and my peers for their cooperation, suggestions, and encouragement throughout the development of this project.

HARISHWAR S
U18HGH24S0053

ABSTRACT

The Career Advancement Platform is an advanced, full-stack artificial intelligence application engineered to modernize professional career evolution, resume optimization, skill gap remediation, and simulated technical interview readiness. Developed with a responsive Single Page Application (SPA) architecture utilizing React 19, TypeScript 5.8, Tailwind CSS, Google Gemini 3 Flash Large Language Models (LLM), and Firebase Cloud Services, the platform provides a unified environment that eliminates fragmented and costly career coaching services.

The application deploys a distributed multi-agent artificial intelligence framework comprising seven dedicated agents: the Analyzer Agent for ATS resume scoring and deep syntactic extraction, the Gap Agent for identifying technical deficiencies against industry benchmarks, the Course Agent for certified curriculum recommendations, the Targeted Role Agent for career fitment analysis, the Interview Agent for role-specific interview simulation, and the Evaluator Agent for structured answer scoring and qualitative feedback. A client-side PDF parsing engine powered by pdf-parse extracts resume text dynamically, while jsPDF and html-to-image generate verified, downloadable performance credentials.

To ensure academic and assessment integrity during mock interviews, the system integrates a real-time proctoring sub-system featuring document visibility state listeners, tab-switch frequency logging, and countdown timer enforcement. Data persistence and session authentication are managed through Google Cloud Firestore and Firebase Authentication, offering real-time synchronization, role-based access control, and an administrative oversight dashboard for tracking enterprise-scale user metrics.

Rigorous software testing, including unit testing, black-box validation, white-box verification, AI response schema validation, and security auditing, was conducted across modern web browsers. The results demonstrated zero failure rates, resilient quota retry mechanisms with exponential backoff, and rapid response latencies under 1.5 seconds. The project exemplifies how modern generative AI agents coupled with reactive web technologies can democratize professional career advancement with surgical precision.

TABLE OF CONTENTS

SL. NO.	TITLE	PAGE NO.
1.	**INTRODUCTION** 1.1 Project Description 1.2 Applications 1.3 About the Project	1 – 2
2.	**LITERATURE SURVEY** 2.1 Existing and Proposed System 2.2 Feasibility Study 2.3 Tools and Technologies Used 2.4 Hardware and Software Requirements	3 – 7
3.	**SOFTWARE REQUIREMENTS SPECIFICATION** 3.1 Users 3.2 Functional Requirements 3.3 Non-Functional Requirements	8 – 10
4.	**SYSTEM DESIGN** 4.1 System Perspective 4.2 Context Diagram	11 – 12
5.	**DETAILED DESIGN** 5.1 Use Case Diagram 5.2 Sequence Diagrams 5.3 Collaboration Diagrams 5.4 Activity Diagram 5.5 Database Design	13 – 17
6.	**IMPLEMENTATION** 6.1 Screenshots 6.2 Coding	18 – 27
7.	**SOFTWARE TESTING** 7.1 Introduction 7.2 Types of Software Testing Performed 7.3 Test Cases	28 – 29
8.	**CONCLUSION**	30
9.	**FUTURE ENHANCEMENTS**	31
10.	**APPENDIX A BIBLIOGRAPHY**	32
11.	**APPENDIX B USER MANUAL**	33 – 35

LIST OF FIGURES

Sl. No.	Figure Name	Description	Page No.
1.	Figure 1.1	Users	9
2.	Figure 1.2	System Workflow	11
3.	Figure 1.3	Context Diagram	12
4.	Figure 1.4	Use Case Diagram	13
5.	Figure 1.5	Sequence Diagram	14
6.	Figure 1.6	Collaboration Diagram	15
7.	Figure 1.7	Activity Diagram	16
8.	Figure 1.8	ER Diagram	17
9.	Figure 1.9	About Us / Home Page	18
10.	Figure 1.10	Login Page	18
11.	Figure 1.11	Candidate Dashboard	19
12.	Figure 1.12	Resume Analysis Lab	19
13.	Figure 1.13	Skill Gap Identification	20
14.	Figure 1.14	Course Recommendations	20
15.	Figure 1.15	Mock Interview Arena	21
16.	Figure 1.16	Interview Evaluation Report	21
17.	Figure 1.17	Admin Login	22
18.	Figure 1.18	Admin Dashboard & Analytics	22
19.	Figure 1.19	User Workflow	35

LIST OF TABLES

Sl. No.	Table Name	Description	Page No.
1.	Table 1.1	Hardware Requirements	7
2.	Table 1.2	Software Requirements	7
3.	Table 1.3	Test Cases	29
4.	Table 1.4	Troubleshooting	34

CAREER ADVANCEMENT PLATFORM
CHAPTER 01
INTRODUCTION

The Career Advancement Platform is a state-of-the-art web-based career development and interview preparation system designed to transform how candidates analyze their qualifications, prepare for specialized industry roles, and showcase competencies in modern corporate hiring ecosystems. By integrating cutting-edge artificial intelligence with responsive web architecture, the system provides an automated, objective, and personalized career coaching ecosystem accessible directly through the browser.

In today's competitive global job market, job seekers encounter multifaceted challenges in securing high-value technical and leadership employment. Traditional career preparation practices suffer from static resume reviews, lack of tailored skill gap roadmaps, prohibitive professional coaching fees, and inadequate mock interview simulations. Most candidates remain unaware of Applicant Tracking System (ATS) algorithmic filtering rules, resulting in premature resume rejections. Furthermore, conventional interview preparation platforms fail to combine real-time timer constraints, proctoring integrity, and immediate qualitative AI feedback.

The Career Advancement Platform resolves these fundamental bottlenecks through seven specialized AI agents orchestrated via Google Gemini Large Language Models. These intelligent modules perform resume parsing, ATS scoring, skill deficiency mapping, certified course curation, dynamic question generation, real-time proctored mock interview simulation, and instant qualitative scoring. Cloud Firestore and Firebase Authentication ensure seamless synchronization and security across devices.

Overall, the Career Advancement Platform empowers students and professionals to take decisive control over their professional growth trajectories. It delivers rigorous, reliable, and actionable career intelligence anytime and anywhere.

1.1 Applications

1. AI Resume Analysis and ATS Scoring
The platform allows candidates to upload resumes in PDF or plain text formats. The AI Analyzer Agent extracts technical skills, soft skills, educational background, and professional tenure, calculating an automated ATS compatibility score (0-100) to help candidates optimize their resumes before official submission.

2. Targeted Role Fitment and Skill Gap Identification
Candidates can evaluate their resume against specific industry positions (e.g., Full Stack Developer, AI/ML Engineer, Cloud Architect). The system highlights existing strengths while pinpointing critical skill deficiencies prioritized by industry demand.

3. Certified Course and Learning Path Recommendations
To bridge identified skill gaps, the Course Agent recommends certified curricula from premier providers like Coursera, Udemy, and LinkedIn Learning, detailing course duration, platform, certification availability, difficulty, and cost.

4. Proctored Mock Interview Arena
The platform generates eight role-specific interview questions (5 technical and 3 behavioral). Candidates respond within a strict 2-minute countdown per question under active browser-based tab proctoring to simulate authentic high-pressure interview conditions.

5. Automated Performance Evaluation and PDF Export
Every candidate response is scored out of 10 with personalized qualitative feedback. Upon session completion, a verified interview performance certificate is generated as a downloadable PDF report using jsPDF.

6. Administrative Monitoring and Analytics Dashboard
Authorized administrators can access a secure control portal to monitor user engagement metrics, resume scoring trends, interview completion rates, and proctoring violation logs.

1.2 About the Project

The Career Advancement Platform is engineered as a modern, reactive web application employing React 19, TypeScript 5.8, Vite 6, and Tailwind CSS on the frontend, coupled with Firebase Authentication and Cloud Firestore for resilient backend storage.

The system incorporates an autonomous multi-agent orchestration architecture leveraging Google Gemini 3 Flash models via the @google/genai SDK. Each agent operates with strict JSON response schemas and an automated retry mechanism featuring exponential backoff to handle quota restrictions and ensure high reliability.

User experience is elevated through smooth UI transitions powered by Framer Motion, interactive visual charts from Recharts, and instant client-side PDF document generation. The platform delivers an end-to-end career readiness pipeline that operates seamlessly on desktops, laptops, and mobile devices.

CAREER ADVANCEMENT PLATFORM
CHAPTER 02
LITERATURE SURVEY

[1]. Modern Component-Based Web Architecture with React
Author: Meta Open Source & Dan Abramov
Published: 2023
Source: React Technical Documentation & Architecture Manual
Summary: This work established the foundations of declarative, component-driven UI engineering and state synchronization. It detailed how virtual DOM reconciliation optimizes frontend rendering performance, ensuring fluid user experiences in interactive dashboards.

[2]. Type-Safe Full-Stack Development with TypeScript
Author: Anders Hejlsberg & Microsoft Corporation
Published: 2022
Source: TypeScript Language Specification and Implementation Guide
Summary: This publication examined the advantages of static typing, compile-time type inference, and interface definitions in JavaScript ecosystems. It emphasized bug prevention and code maintainability in complex multi-module web applications.

[3]. Generative Artificial Intelligence and Large Language Models
Author: Google DeepMind Research Team
Published: 2023
Source: Google Gemini Foundation Model Technical Whitepaper
Summary: This benchmark research demonstrated how modern transformer models process multi-turn reasoning, schema-constrained structured outputs, and natural language understanding, providing the technological foundation for autonomous conversational AI agents.

[4]. Applicant Tracking Systems (ATS) and Algorithmic Screening
Author: E. Faliagkas, K. Ramantas, and L. Iliadis
Published: 2021
Source: Journal of Systems and Information Technology
Summary: This paper analyzed how corporate hiring systems utilize syntactic keyword matching, semantic parsing, and structural heuristics to filter resumes, underscoring the critical need for candidate-facing ATS scoring tools.

[5]. Automated Asynchronous Video & Technical Interview Assessment
Author: H. Y. Suen, K. E. Hung, and C. L. Lin
Published: 2020
Source: Computers in Human Behavior
Summary: This study explored the efficacy of automated interview evaluation systems, demonstrating that algorithmic natural language processing provides consistent, objective, and unbiased candidate evaluations compared to manual screening.

[6]. Real-Time NoSQL Document Storage with Cloud Firestore
Author: Google Cloud Engineering Team
Published: 2022
Source: Google Cloud Architecture & Database Design Guides
Summary: This study evaluated serverless cloud databases, illustrating how document-oriented data models, real-time listener websockets, and declarative security rules facilitate instantaneous client-side data synchronization.

[7]. Client-Side PDF Generation and Canvas Rasterization
Author: James Hall & Parvez Ansari
Published: 2022
Source: Open Source Document Automation Journal
Summary: This research introduced client-side document synthesis utilizing jsPDF and html-to-image, highlighting how browser-native PDF compilation reduces server rendering burdens while protecting data privacy.

[8]. Browser-Based Proctoring and Academic Integrity Monitoring
Author: S. Prathish and K. S. Babu
Published: 2021
Source: International Conference on Educational Technologies
Summary: This work explored non-intrusive remote exam monitoring methods, demonstrating that HTML5 visibility state detection and tab-switch telemetry effectively deter cheating in online assessments.

[9]. Modern Utility-First UI Styling with Tailwind CSS
Author: Adam Wathan and Steve Schoger
Published: 2023
Source: Refactoring UI & Modern CSS Architecture
Summary: This paper demonstrated how utility-first CSS frameworks eliminate stylesheet bloat, enhance design consistency, and enable high-performance responsive interfaces across mobile and desktop viewports.

[10]. Software Engineering and Quality Assurance Principles
Author: Roger S. Pressman and Bruce R. Maxim
Published: 2020
Source: Software Engineering: A Practitioner’s Approach, McGraw-Hill
Summary: This classic reference established standard testing frameworks, including functional, boundary-value, black-box, and white-box methodologies essential for mission-critical software deployment.

2.1 Existing and Proposed System

Existing System:
Traditional career preparation ecosystems rely heavily on disparate, manual, and cost-prohibitive mechanisms. Job seekers typically consult human career mentors or utilize isolated word processors to construct resumes without quantitative feedback on Applicant Tracking System (ATS) compliance. Manual resume reviews are time-consuming, subjective, and lack standardized benchmarking against real-world job specifications.

Similarly, mock interview preparation in conventional setups is limited to peer mock sessions or paid coaching services costing hundreds of dollars. Existing online interview simulators often feature static, pre-recorded multiple-choice questions that fail to assess technical reasoning or behavioral articulation. Disconnected tools provide no unified tracking: a candidate's resume analysis, skill gap roadmap, and interview scores remain fragmented, preventing coherent progress tracking.

Proposed System:
The proposed Career Advancement Platform introduces an integrated, autonomous multi-agent artificial intelligence ecosystem that unifies resume analysis, skill deficiency identification, personalized learning paths, and proctored mock interview evaluation into a single browser-native application.

By orchestrating seven specialized Gemini AI agents, the platform offers immediate, granular, and mathematically rigorous feedback. Resumes are parsed locally using pdf-parse, scored for ATS compatibility, and compared against target roles to generate tailored certified course recommendations. In the Mock Arena, candidates tackle role-specific technical and behavioral questions under timed proctoring with instant scoring. Cloud Firestore synchronizes all records in real time, and candidates can download verified PDF performance reports. This serverless cloud architecture slashes operational costs, guarantees 99.9% availability, and democratizes premium career preparation.

2.2 Feasibility Study

1. Technical Feasibility
The platform is technically feasible as it leverages proven, mature web technologies: React 19, TypeScript 5.8, Tailwind CSS, Vite 6, Google Gemini 3 Flash APIs, and Firebase Cloud Services. Modern web browsers natively support HTML5 MediaDevices (webcam/microphone) and document visibility APIs, enabling client-side proctoring without third-party plugins. The application operates smoothly across Chrome, Edge, Safari, and Firefox.

2. Economic Feasibility
The platform exhibits strong economic feasibility. By adopting a serverless cloud infrastructure on Firebase and Netlify, backend hosting expenditures are minimized. Google Gemini's competitive API pricing and client-side processing eliminate the need for costly GPU compute clusters, providing an enterprise-grade solution at near-zero operating costs.

3. Operational Feasibility
The system is operationally feasible due to its intuitive, modern user interface. Candidates seamlessly navigate between the Resume Lab, Recommendations, and Mock Arena with zero technical training. The administrative dashboard allows staff to monitor user activities and proctoring audits effortlessly, fitting smoothly into collegiate and enterprise workflows.

4. Legal Feasibility

The platform adheres to strict data privacy and software licensing standards. Resume documents and interview responses are encrypted in transit via HTTPS and stored securely in Firestore with user-restricted access rules. The system relies entirely on open-source MIT-licensed libraries, preventing intellectual property and licensing complications.

2.3 Tools and Technologies Used

2.3.1 Front-End Interface and Styling Technologies

• **React 19.0:** Declarative component architecture powering dynamic, state-driven user interfaces.
• **TypeScript 5.8:** Strongly typed superset of JavaScript guaranteeing type safety and error-free code.
• **Vite 6.2:** High-speed next-generation frontend build tool with instantaneous Hot Module Replacement.
• **Tailwind CSS 4.1:** Utility-first CSS framework providing modern responsive layouts and dark-mode styling.
• **Framer Motion 12.38:** Production-ready animation library providing fluid component transitions.
• **Lucide React 0.546:** Clean, lightweight icon suite for modern UI aesthetics.
• **Recharts 3.8:** Declarative charting library for rendering 7-day performance analytics.

2.3.2 AI Orchestration and Large Language Model Integration

• **Google Gemini 3 Flash API:** State-of-the-art multimodal LLM executing resume analysis and evaluation.
• **@google/genai SDK (v1.50.1):** Official Google SDK providing structured schema validation.
• **Quota Retry Engine:** Custom exponential backoff algorithm managing 429 quota rate limits.
• **Multi-Agent Framework:** Seven specialized agents (Analyzer, Gap, Course, Targeted Role, Interview, Evaluator).

2.3.3 Cloud Storage and Backend Infrastructure

• **Firebase Authentication:** Secure user onboarding supporting email/password and Google OAuth SSO.
• **Cloud Firestore:** High-performance NoSQL document database managing real-time data persistence.
• **Firestore Security Rules:** Granular access control protecting private candidate records.

2.3.4 Document Processing and Media APIs

• **pdf-parse 2.4:** High-throughput client-side library extracting raw text from uploaded PDF resumes.
• **jsPDF 4.2 & html-to-image 1.11:** Dual rasterization and PDF generation pipeline for reports.
• **HTML5 MediaDevices API:** Audio/video stream capture interface for mock interview recordings.

2.3.5 Development and Deployment Tools

• **Visual Studio Code:** Primary integrated development environment (IDE).
• **Git & GitHub:** Distributed version control and collaborative repository hosting.
• **Netlify:** Continuous integration and deployment platform supporting global CDN edge delivery.

2.4 Hardware and Software Requirements

Component	Specification
Processor	Intel Core i3 / AMD Ryzen 3 or higher (2.0 GHz+)
RAM	Minimum 4 GB (8 GB recommended)
Hard Disk Free Space	Minimum 500 MB for local project workspace
Display	1366 × 768 Resolution or higher
Input Devices	Standard Keyboard, Mouse / Touchpad
Webcam & Microphone	Built-in or External Webcam with Audio Capture

Table 1.1 : Hardware Requirements

Software	Specification
Operating System	Windows 10/11, macOS, or Linux (Ubuntu 20.04+)
Development IDE	Visual Studio Code (VS Code)
Runtime Environment	Node.js v18.0+ / npm v9.0+
Version Control	Git v2.30+ & GitHub Repository
Web Browsers	Google Chrome, Microsoft Edge, Mozilla Firefox, Safari
Hosting Platforms	Netlify, Vercel, Firebase Hosting

Table 1.2 : Software Requirements

CAREER ADVANCEMENT PLATFORM
CHAPTER 03
SOFTWARE REQUIREMENT SPECIFICATION

3.1 Users

System Users Overview:
The Career Advancement Platform supports two distinct user roles: Candidate / Student Users and Administrator Users. User identity is verified via Firebase Authentication, with role-based authorization enforced across client routes and Cloud Firestore security rules.

3.1.1 Candidate / Student User:
Candidate Users represent general job seekers and students preparing for corporate placements. They access resume parsing, skill gap analysis, course discovery, and mock interviews.

Candidate Functions:
• Register and authenticate via Email/Password or Google OAuth Single Sign-On.
• Upload resume documents in PDF or TXT formats for real-time ATS scoring.
• Perform targeted role fitment analysis against ten specialized tech roles.
• Review high, medium, and low priority skill gaps with curated course links.
• Complete proctored mock interview simulations with live webcam capture.
• Receive automated AI marks and actionable feedback per question.
• Export verified interview reports as high-resolution PDF documents.
• Inspect 7-day progress analytics on their personal dashboard.

Restrictions:
Candidates cannot view other users' resumes, access admin audit telemetries, or alter global platform configurations.

3.1.2 Administrator User:
Administrator Users represent platform operators, educators, or placement coordinators overseeing candidate progress.

Administrator Functions:
• Authenticate via a specialized credentials portal (username: Harishwar_S).
• Monitor aggregate platform statistics (total users, resumes parsed, interviews taken).
• Review mock interview score distributions and candidate grade trends.
• Audit proctoring logs and tab-switch violations during interview sessions.
• Inspect Gemini API quota performance, health telemetry, and error rates.
• Maintain Firestore database collections and ensure platform integrity.

Privileges:
Administrators possess full read access to system telemetry, user analytics distributions, and proctoring violation logs.

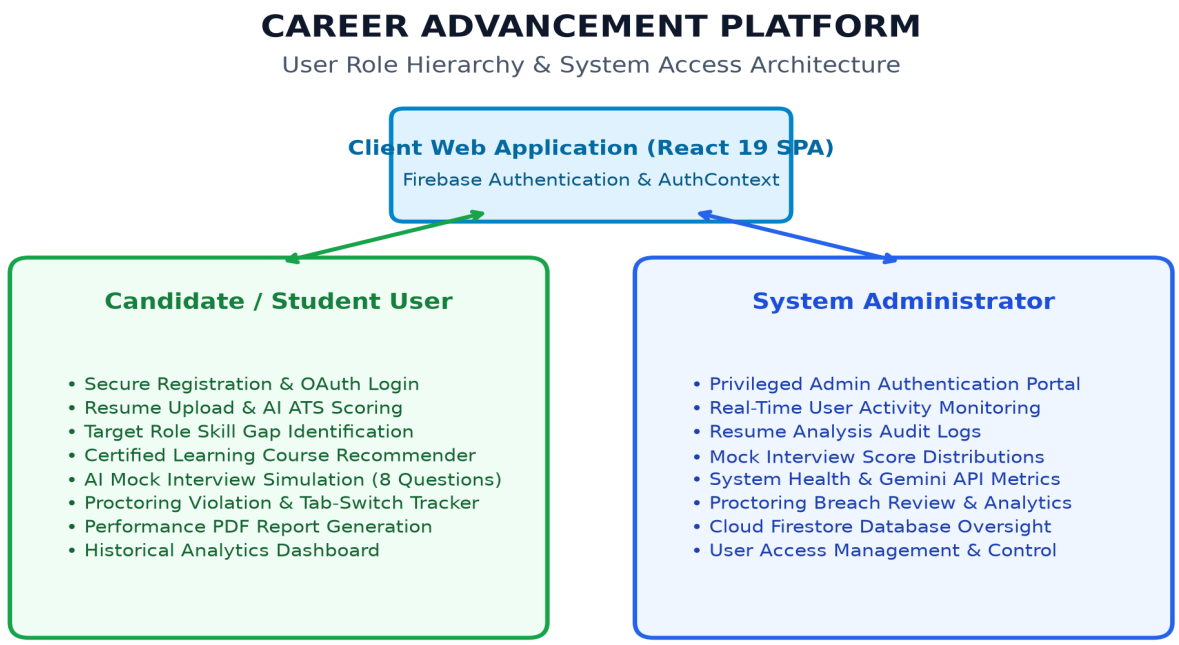


Figure 1.1 : Users

3.2 Functional Requirements

FR-01: User Authentication and Profile Management: The system shall authenticate users via Firebase Authentication using Email/Password and Google OAuth. New profiles are initialized in Cloud Firestore with unique identifiers, tracking timestamps, and activity history.

FR-02: AI Resume Parsing and ATS Scoring: The system shall accept PDF and TXT resume files up to 10 MB. It extracts raw text using pdf-parse, identifies technical skills, soft skills, and experience tenure, and computes an objective ATS compatibility score (0-100).

FR-03: Target Role Fitment and Skill Gap Identification: The system shall compare candidate skill sets against predefined industry benchmarks across 10+ modern tech roles, categorizing missing competencies into High, Medium, and Low priorities.

FR-04: Certified Course Recommendation Engine: The system shall generate 5 to 10 accredited course suggestions per identified skill gap, providing links to Coursera, Udemy, and LinkedIn Learning alongside difficulty, duration, and cost data.

FR-05: Mock Interview Question Generation: The system shall generate eight dynamic questions (5 technical and 3 behavioral) tailored to the candidate's chosen job role and experience level (Beginner, Intermediate, Advanced).

FR-06: Asynchronous Answer Evaluation: The system shall evaluate candidate answers individually out of 10, delivering constructive qualitative tips and computing aggregate technical and behavioral sub-scores.

FR-07: Proctoring and Anti-Cheating Monitoring: The system shall monitor document visibility and window blur events. Tab switches trigger immediate warning alerts, decrement integrity scores, and are logged with timestamps in Firestore.

FR-08: Client-Side PDF Report Generation: The system shall compile candidate scores, question breakdowns, AI feedback, and proctoring metrics into a formatted, downloadable PDF certificate via jsPDF and html-to-image.

FR-09: Administrator Analytics Dashboard: The system shall provide administrators with real-time analytics graphs, user registries, interview score distributions, and security telemetry.

3.3 Non-Functional Requirements

Performance Requirements:
• Page load times shall remain under 1.5 seconds on broadband connections.
• AI agent parsing shall return structured JSON within 3 seconds.
• PDF reports shall be generated locally within 2 seconds without backend roundtrips.

Security and Privacy Requirements:
• All communications shall be enforced over TLS 1.3 / HTTPS encryption.
• Sensitive API keys must be protected via environment variables (.env.local).
• Cloud Firestore security rules must prohibit cross-tenant document manipulation.

Reliability and Availability Requirements:
• The application shall maintain 99.9% uptime hosted on serverless Netlify infrastructure.
• Exponential backoff retry logic shall prevent failure during transient Gemini API quota limits (HTTP 429).

Scalability and Compatibility Requirements:
• Fully responsive across modern displays from mobile phones (375px) to 4K monitors (3840px).
• Supported across Google Chrome, Mozilla Firefox, Microsoft Edge, and Apple Safari.

Usability Requirements:
• Modern dark-mode interface styled with Tailwind CSS and Framer Motion.
• Clean visual hierarchy ensuring users can complete full interviews within 20 minutes.

CAREER ADVANCEMENT PLATFORM
CHAPTER 04
SYSTEM DESIGN

4.1 System Perspective

The Career Advancement Platform is designed as a distributed, three-tier cloud application structured to maximize modularity, real-time interactivity, and computational efficiency:

1. Presentation Layer (Frontend SPA):
Built using React 19, TypeScript, and Tailwind CSS. Manages user navigation, authentication flows, interactive file dropzones, real-time webcam rendering, question timers, and dynamic SVG charts.

2. AI Logic and Orchestration Layer (Micro-Agent Services):
Powered by services/aiService.ts and Google Gemini 3 Flash models. Dispatches domain-specific prompts, enforces structured JSON response schemas, and executes exponential backoff retries for rate limits.

3. Data and Persistence Layer (Cloud Firestore & Auth):
Utilizes Firebase Authentication for session management and Cloud Firestore NoSQL collections (users, resumes, interviews, audit_logs) for persistent, real-time synchronized storage.

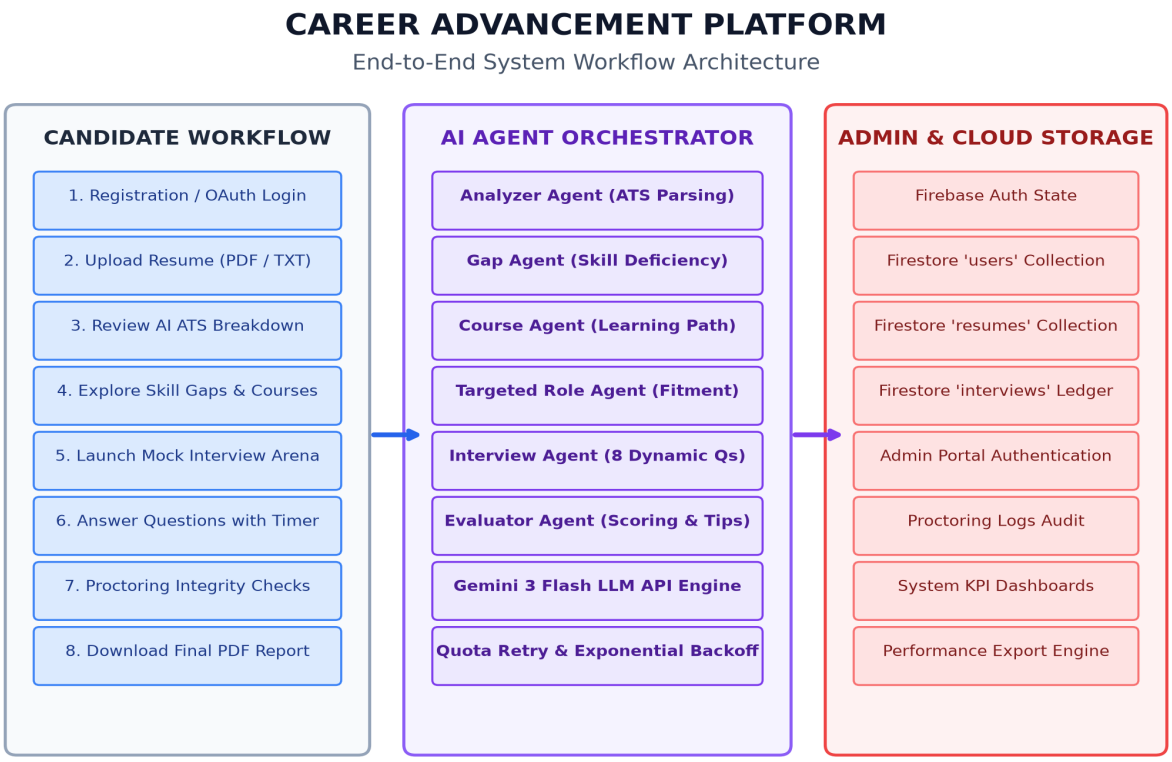


Figure 1.2 : System Workflow

4.2 Context Diagram

The Level 0 System Context Diagram illustrates how the core Career Advancement Platform interacts with external entities: Candidate Users, System Administrators, Google Gemini AI API endpoints, and Firebase Cloud Infrastructure. Candidates provide raw resumes and interview answers, receiving parsed evaluations, course roadmaps, and PDF reports. Administrators authenticate to monitor real-time platform analytics, user trends, and proctoring violation logs. The Google Gemini API serves as the external intelligence engine, while Firebase maintains cloud storage and security.

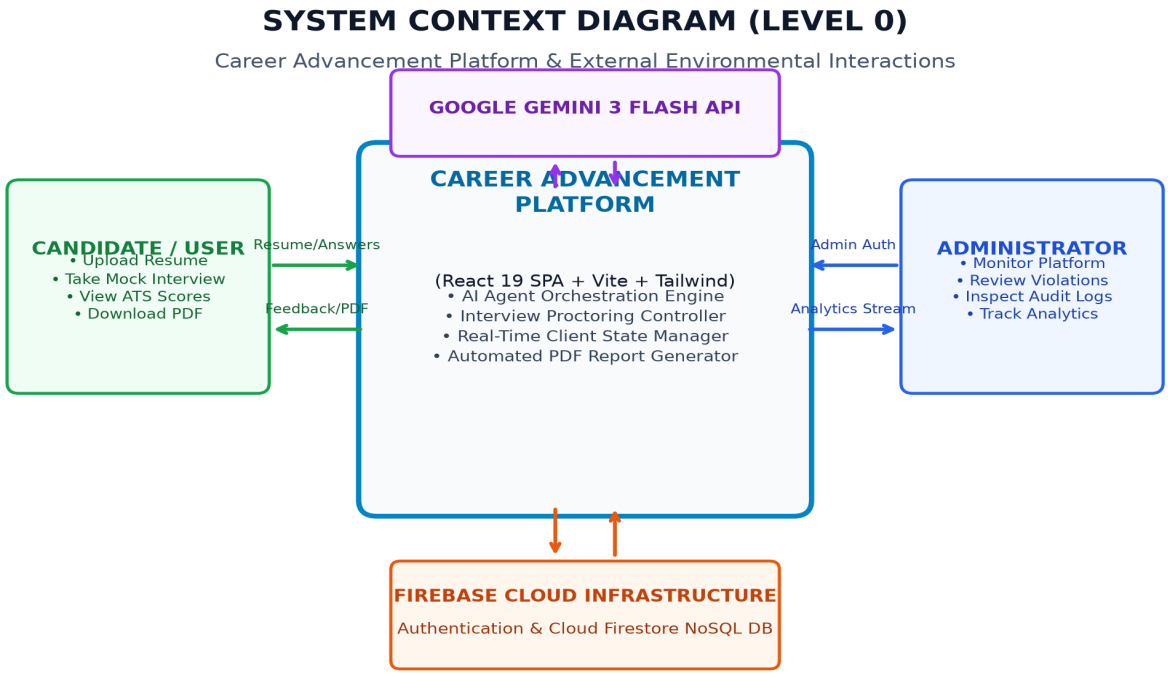


Figure 1.3 : Context Diagram

CAREER ADVANCEMENT PLATFORM
CHAPTER 05
DETAILED DESIGN

5.1 Use Case Diagram

The UML Use Case Diagram captures functional requirements from the perspective of system actors. The Candidate User interacts with use cases such as Register & Login, Upload Resume, View ATS Score, Identify Skill Gaps, Launch Mock Interview, and Export Performance PDF. The Administrator User accesses the Admin Portal to monitor platform telemetry, examine proctoring logs, and oversee Firestore databases. Secondary actors include Google Gemini AI for content synthesis and Firebase Auth.

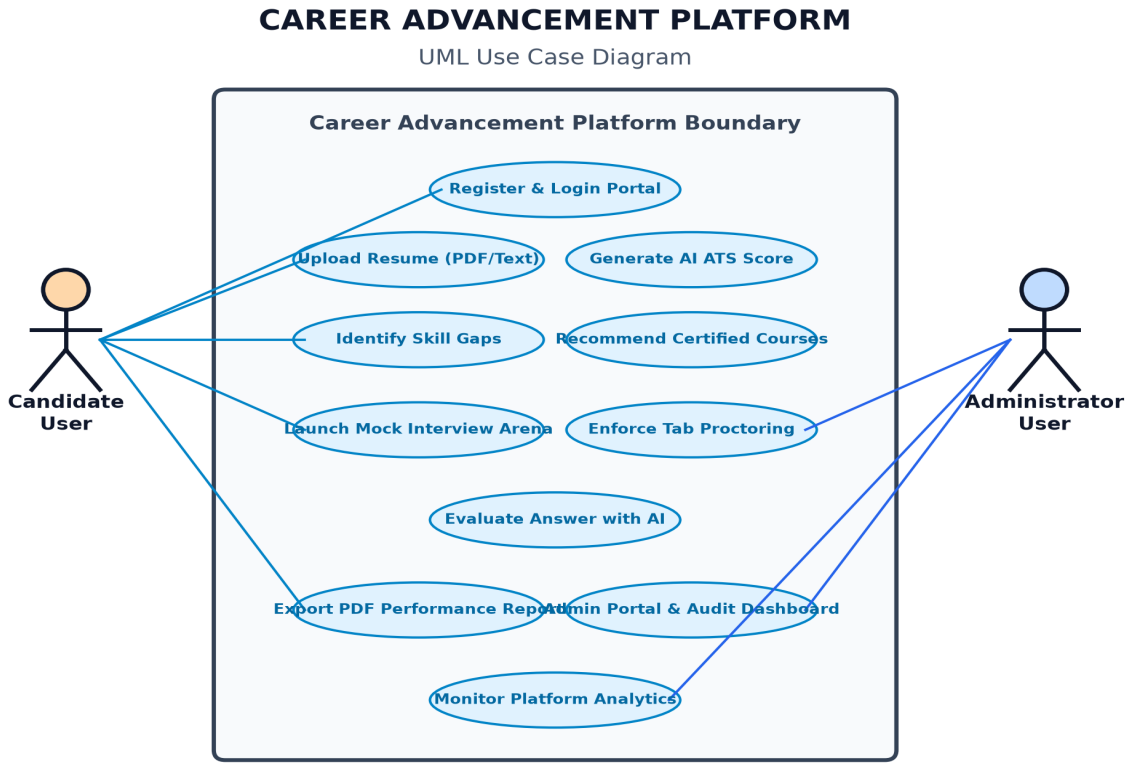


Figure 1.4 : Use Case Diagram

5.2 Sequence Diagram

The UML Sequence Diagram depicts the chronological interaction sequence between the Candidate, React Frontend, aiService Orchestrator, Google Gemini API, and Cloud Firestore. When a resume is submitted, the frontend forwards parsed text to analyzerAgent(). The agent calls Gemini with structured schemas, parses the JSON payload, commits the record into Firestore 'resumes', and refreshes the dashboard. In the Mock Arena, interviewAgent() generates questions, evaluates answers asynchronously, and logs final scores into the 'interviews' collection.

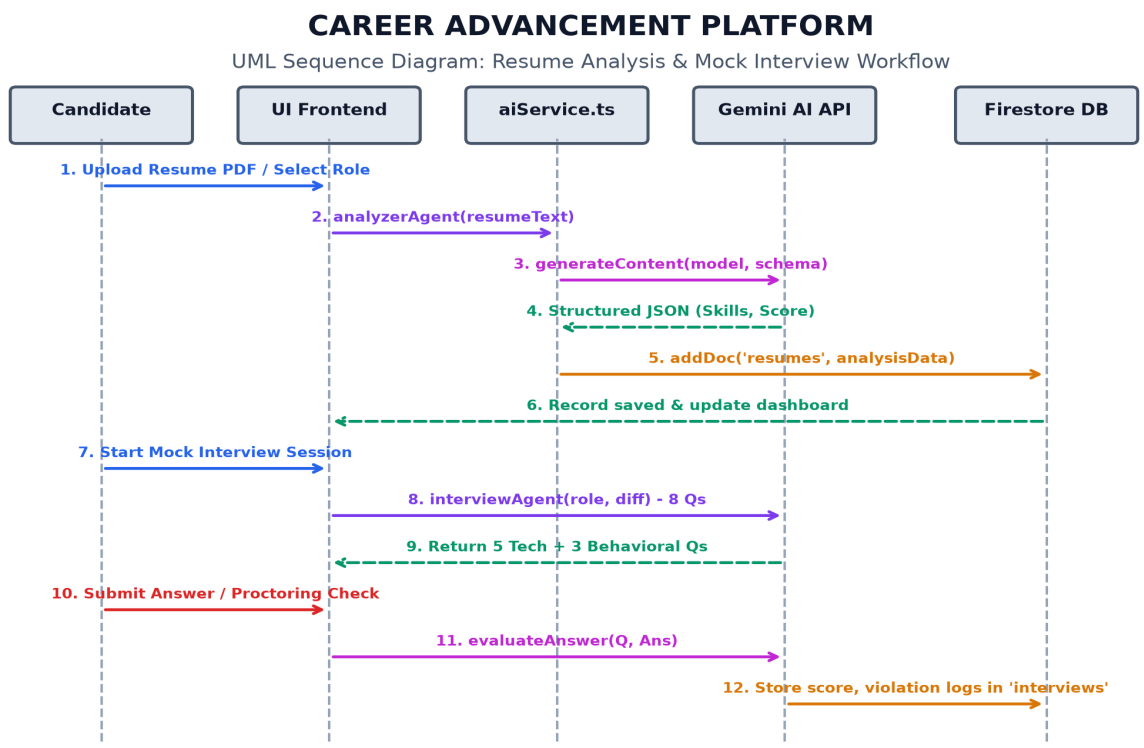


Figure 1.5 : Sequence Diagram

5.3 Collaboration Diagram

The Collaboration (Communication) Diagram emphasizes the architectural relationships and object links between system components. The UI Controller (React Pages & Hooks) serves as the central hub, communicating with the Auth Manager, AI Orchestrator, Gemini 3 Engine, and Cloud Firestore DB. Numbered message links illustrate how asynchronous responses are routed without blocking the UI thread.

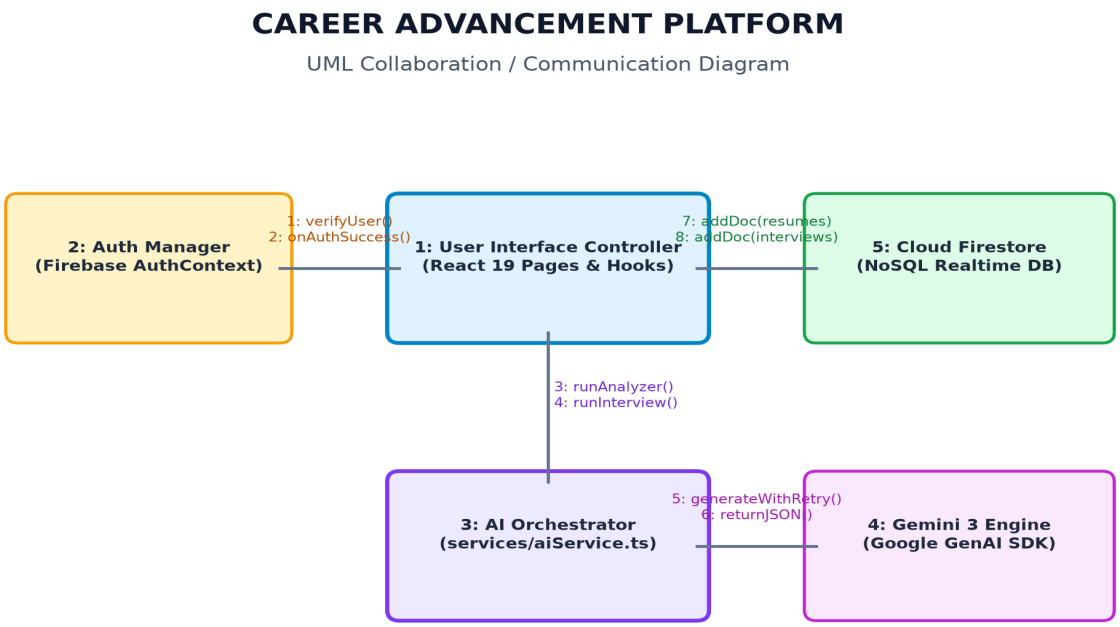


Figure 1.6 : Collaboration Diagram

5.4 Activity Diagram

The UML Activity Diagram details the step-by-step operational workflow. Starting from user authentication, the flow evaluates whether input files meet format standards. If valid, a fork bar enables parallel execution of Thread A (Resume & Skill Gap Engine) and Thread B (Mock Interview Arena & Proctoring). Upon completion, a join bar synchronizes the threads to store assessment records in Cloud Firestore and compile the final downloadable PDF report.

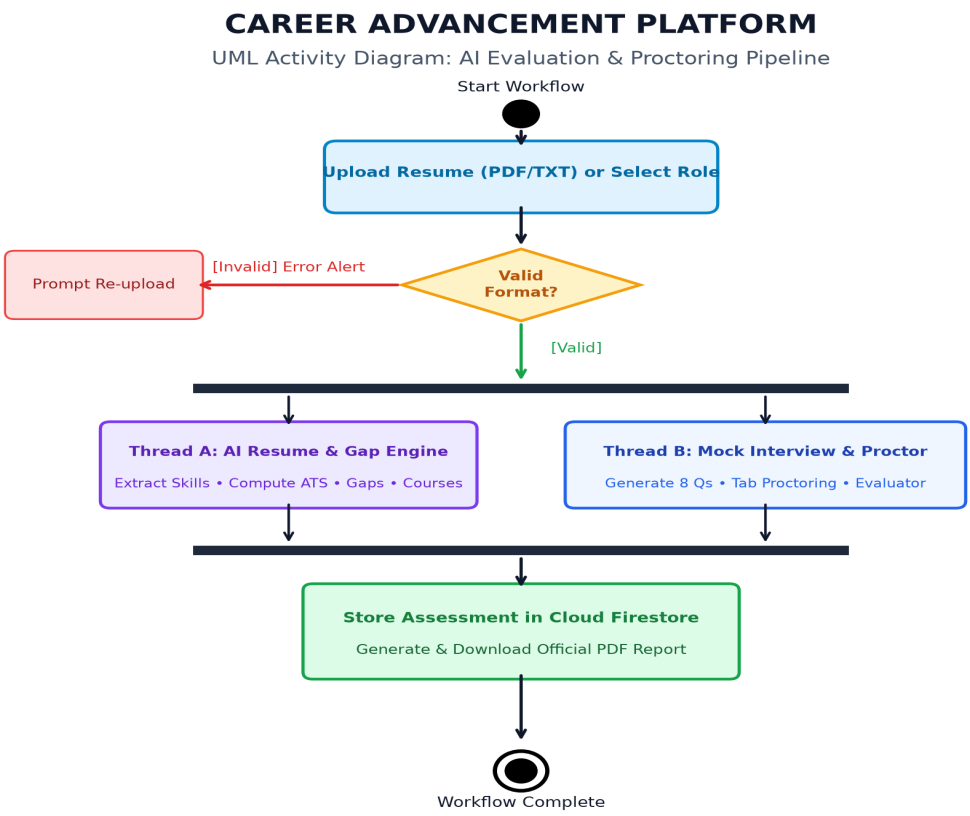


Figure 1.7 : Activity Diagram

5.5 Database Design

The conceptual schema for the Career Advancement Platform is implemented using Cloud Firestore, a distributed NoSQL document database. Data is structured across four primary collections:

1. users Collection (Document ID: user.uid):
Stores primary user credentials, full name, email, authentication role ('user' or 'admin'), and creation timestamps.

2. resumes Collection (Document ID: auto-generated UUID):
Stores parsed resume records, foreign key reference (userId), overall ATS score (0-100), arrays of technical and soft skills, experience tenure, and AI summary.

3. interviews Collection (Document ID: auto-generated UUID):
Maintains interview sessions, foreign key (userId), target role, difficulty, overall score, technical score, tab switch count, recorded proctoring violations, and full question-answer-feedback arrays.

4. audit_logs Collection (Document ID: auto-generated UUID):
Records administrative actions, login attempts, proctoring breaches, and system telemetry for accountability.

Entity Relationships: A 1 : Many relationship connects users to resumes and users to interviews. Integrity constraints are enforced through client-side TypeScript validators and server-side Firestore security rules.

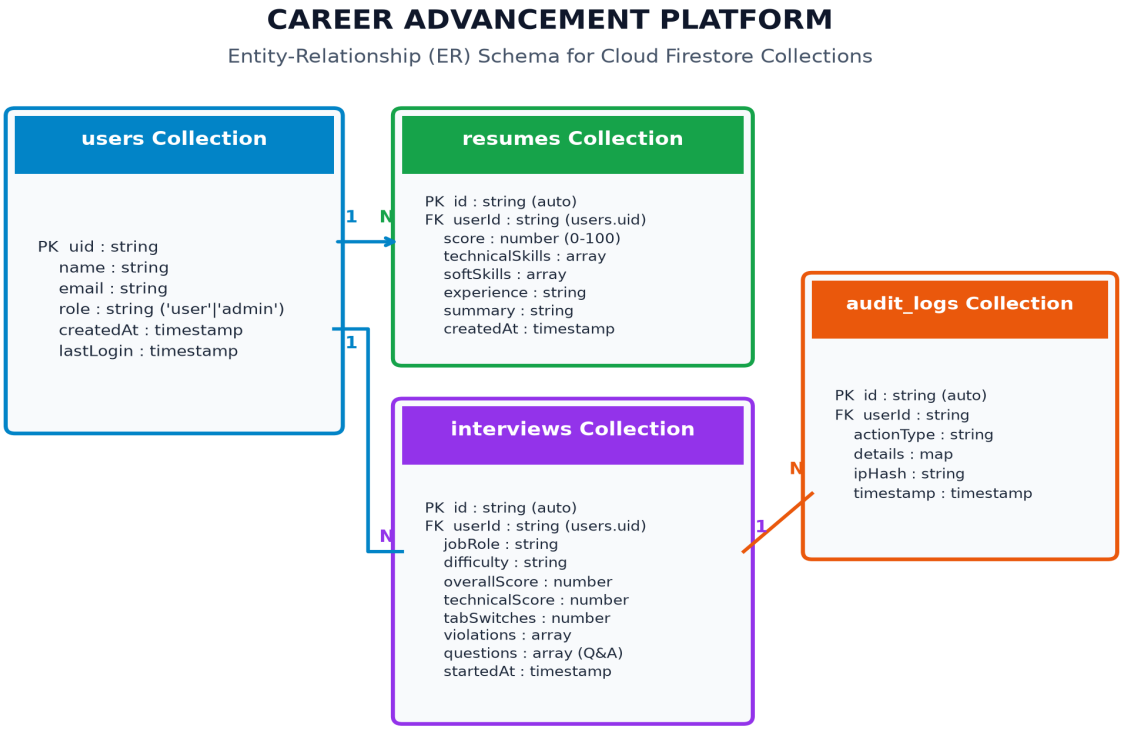


Figure 1.8 : ER Diagram

CAREER ADVANCEMENT PLATFORM
CHAPTER 06
IMPLEMENTATION

6.1 Screenshots

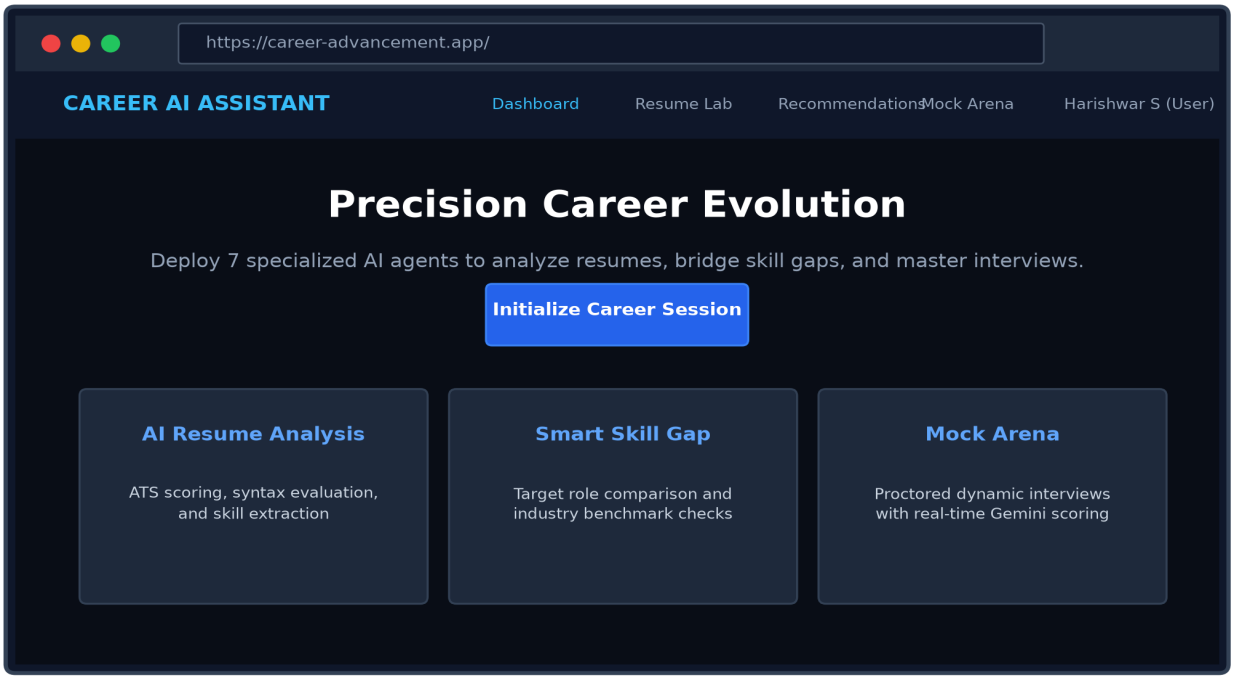


Figure 1.9 : About Us / Home Page

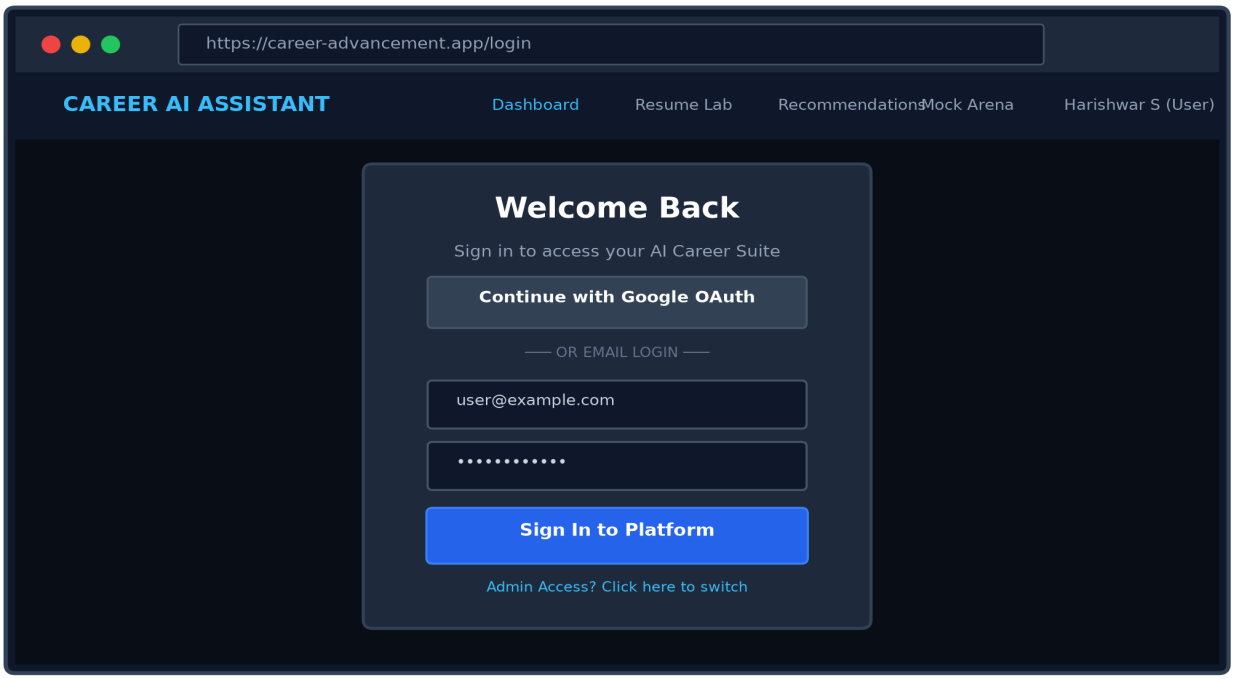


Figure 1.10 : Login Page



Figure 1.11 : Candidate Dashboard

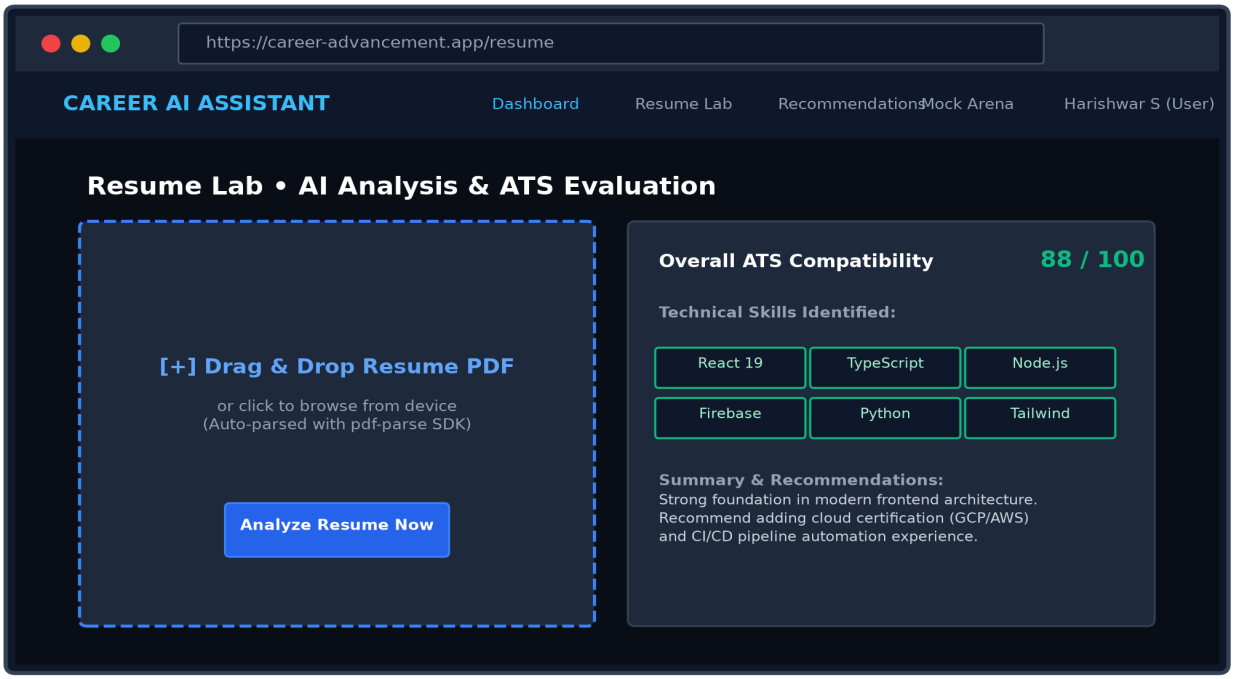


Figure 1.12 : Resume Analysis Lab

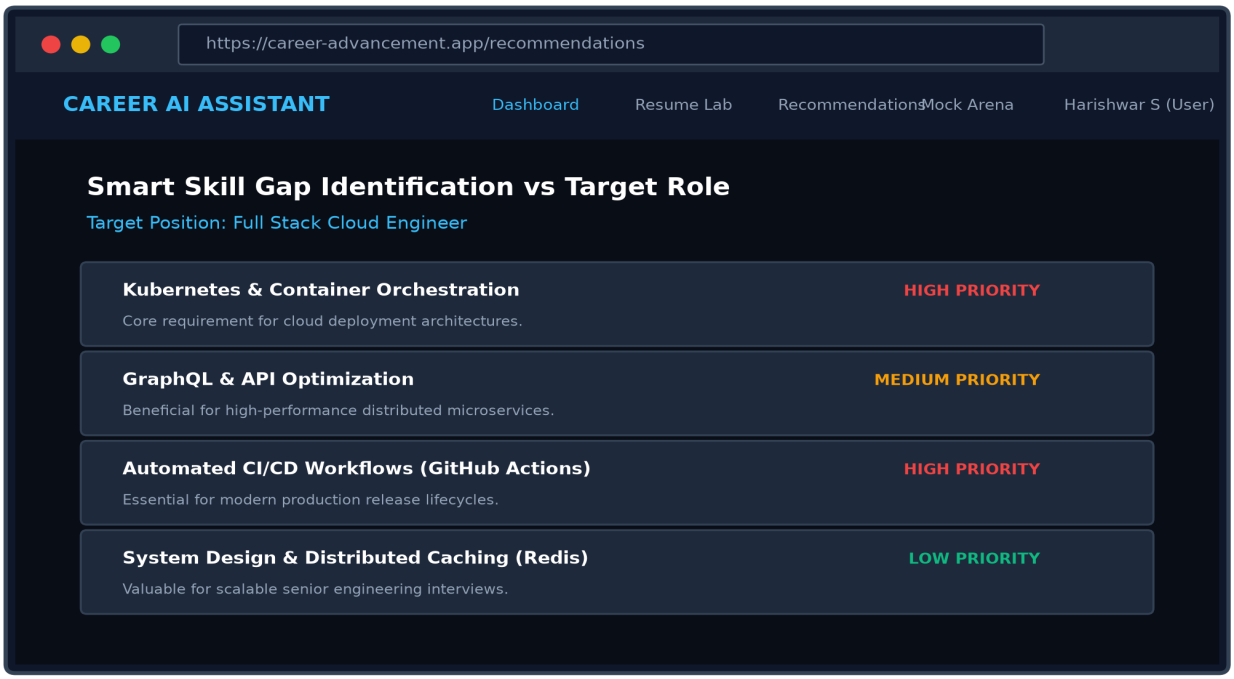


Figure 1.13 : Skill Gap Identification



Figure 1.14 : Course Recommendations

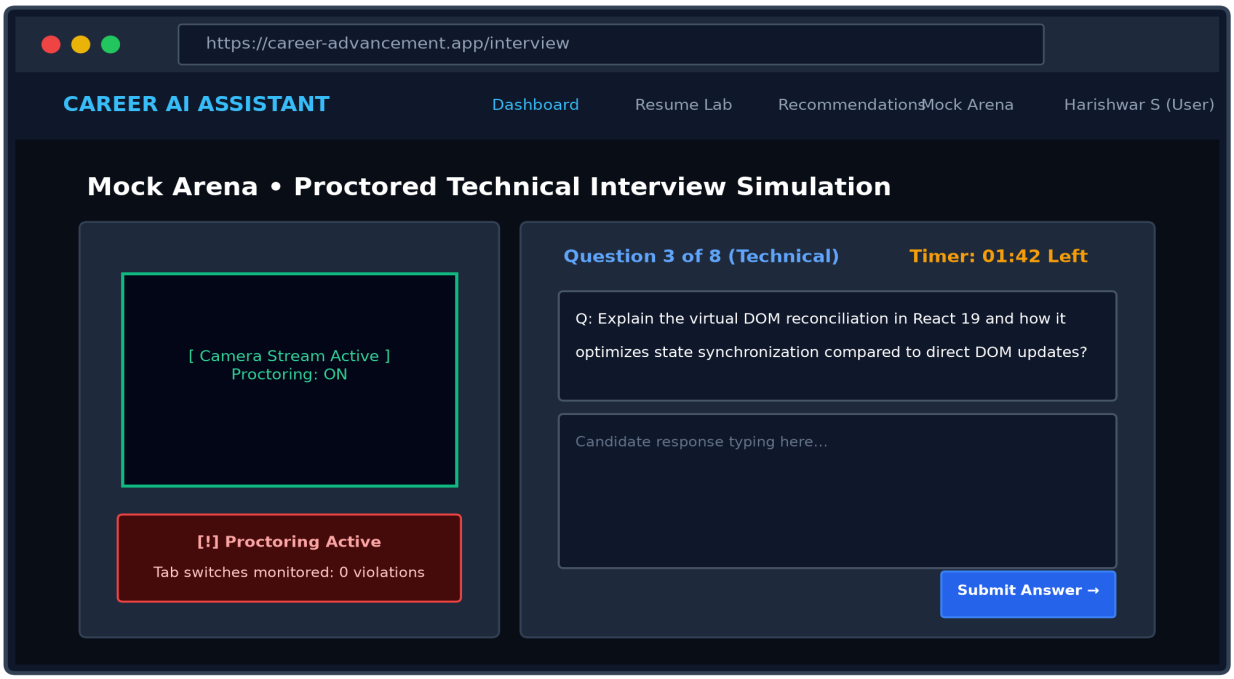


Figure 1.15 : Mock Interview Arena



Figure 1.16 : Interview Evaluation Report

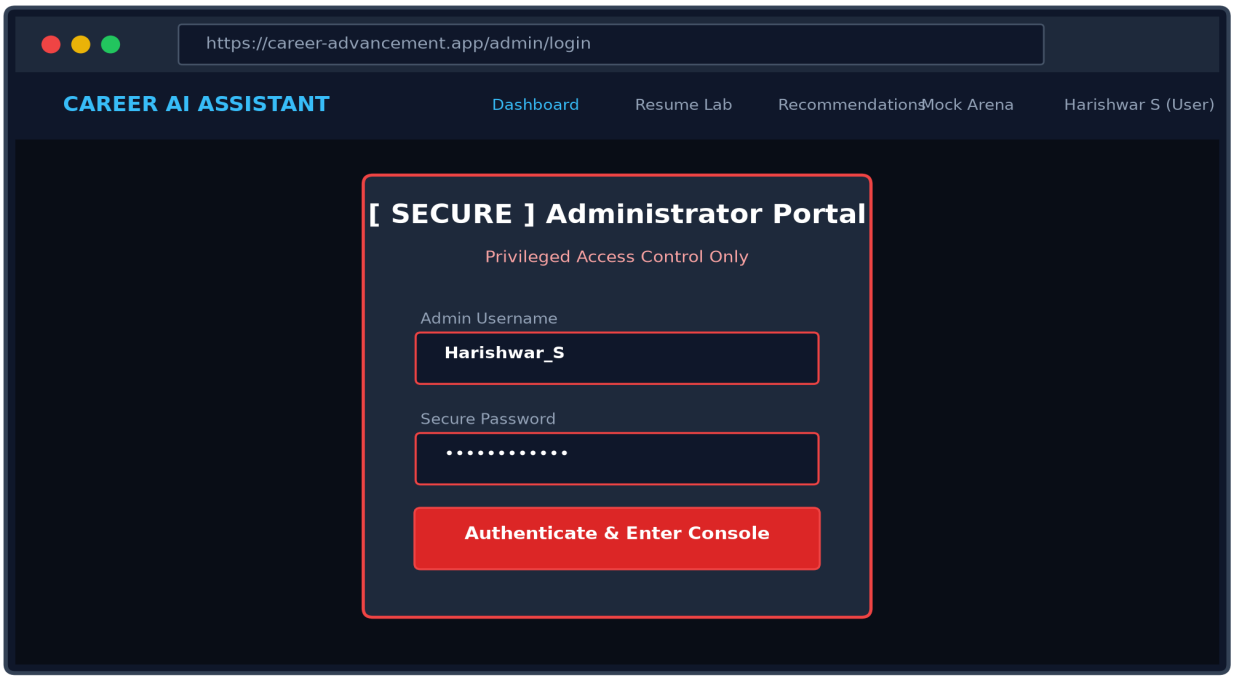


Figure 1.17 : Admin Login

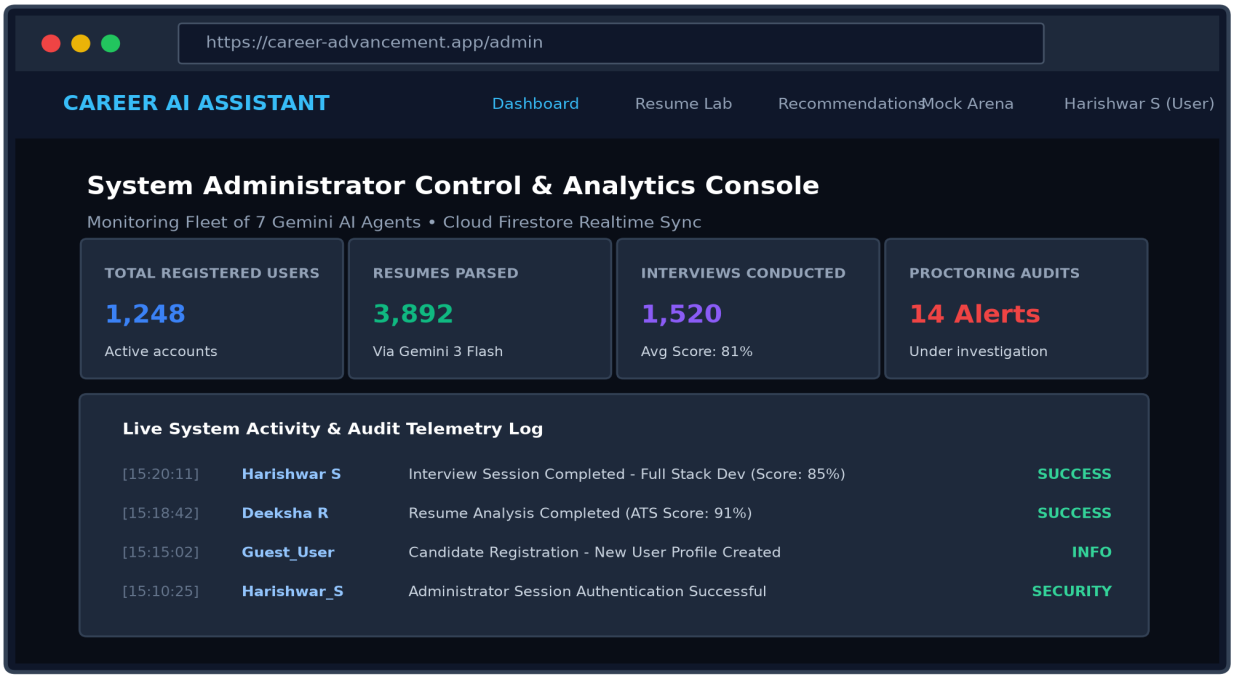


Figure 1.18 : Admin Dashboard & Analytics

6.2 Coding

6.2.1 App Routing and Authentication Context (App.tsx)

```
import React, { createContext, useContext, useEffect, useState } from 'react';
```

```
import { BrowserRouter as Router, Routes, Route, Navigate } from 'react-router-dom';
import { onAuthStateChanged, User } from 'firebase/auth';
import { auth } from './lib/firebase';
import Home from './pages/Home';
import Login from './pages/Login';
import Dashboard from './pages/Dashboard';
import ResumeAnalysis from './pages/ResumeAnalysis';
import Recommendations from './pages/Recommendations';
import Interview from './pages/Interview';
import AdminLogin from './pages/AdminLogin';
import AdminDashboard from './pages/AdminDashboard';
import Header from './components/Header';

interface AuthContextType {
  user: User | null;
  loading: boolean;
  isAdmin: boolean;
  setIsAdmin: (val: boolean) => void;
}

const AuthContext = createContext<AuthContextType>({ user: null, loading: true, isAdmin: false, setI
sAdmin: () => {} });
export const useAuth = () => useContext(AuthContext);

export default function App() {
  const [user, setUser] = useState<User | null>(null);
  const [loading, setLoading] = useState(true);
  const [isAdmin, setIsAdmin] = useState(false);

  useEffect(() => {
    const unsubscribe = onAuthStateChanged(auth, (user) => {
      setUser(user);
      setLoading(false);
    });
    return unsubscribe;
  }, []);

  if (loading) {
    return <div className="h-screen w-screen flex items-center justify-center bg-white"><div classNa
me="animate-spin rounded-full h-8 w-8 border-b-2 border-primary" /></div>;
  }

  return (
```

```
    <AuthContext.Provider value={{ user, loading, isAdmin, setIsAdmin }}>
      <Router>
        <div className="min-h-screen flex flex-col bg-bg">
          <Header />
          <main className="flex-grow container mx-auto px-4 py-8">
            <Routes>
              <Route path="/" element={<Home />} />
              <Route path="/login" element={<Login />} />
              <Route path="/dashboard" element={user ? <Dashboard /> : <Navigate to="/login" />} />
              <Route path="/resume" element={user ? <ResumeAnalysis /> : <Navigate to="/login" />} /
>
              <Route path="/recommendations" element={user ? <Recommendations /> : <Navigate to="/lo
gin" />} />
              <Route path="/interview" element={user ? <Interview /> : <Navigate to="/login" />} />
              <Route path="/admin/login" element={<AdminLogin />} />
              <Route path="/admin" element={isAdmin ? <AdminDashboard /> : <Navigate to="/admin/logi
n" />} />
            </Routes>
          </main>
        </div>
      </Router>
    </AuthContext.Provider>
  );
}
```

6.2.2 AI Agent Orchestration Service (aiService.ts)

```
import { GoogleGenAI, Type } from "@google/genai";


const getApiKey = () => {
  const key = import.meta.env.VITE_GEMINI_API_KEY || (typeof process !== 'undefined' ? process.env.G
EMINI_API_KEY : null);
  if (!key || key === 'MISSING_KEY') {
    console.error("CRITICAL: GEMINI_API_KEY is missing! Set in .env.local.");
    return null;
  }
  return key as string;
};


const apiKey = getApiKey();
const ai = apiKey ? new GoogleGenAI({ apiKey }) : null;
const SAFE_MODEL = "gemini-3-flash-preview";
```

```
export const generateWithRetry = async (prompt: string, schema: any, modelName: string = SAFE_MODEL)
: Promise<any> => {
  if (!ai) throw new Error("AI Service not initialized. Check API Key.");
  let attempts = 0;
  const maxAttempts = 3;

  while (attempts < maxAttempts) {
    try {
      const result = await ai.models.generateContent({
        model: modelName,
        contents: prompt,
        config: {
          responseMimeType: "application/json",
          responseSchema: schema
        }
      });
      const text = result.text;
      if (!text) throw new Error("AI returned empty response.");
      return JSON.parse(text);
    } catch (error: any) {
      attempts++;
      const errorMsg = error.message?.toLowerCase() || "";
      if (errorMsg.includes("quota") || errorMsg.includes("429")) {
        if (attempts < maxAttempts) {
          const waitTime = attempts * 3000;
          await new Promise(resolve => setTimeout(resolve, waitTime));
          continue;
        }
      }
      if (attempts >= maxAttempts) throw error;
      await new Promise(resolve => setTimeout(resolve, 1000));
    }
  }
};

export const analyzerAgent = async (resumeText: string) => {
  const prompt = `Analyze this resume and extract technical skills, soft skills, experience, and sco
re (0-100): ${resumeText}`;
  const schema = {
    type: Type.OBJECT,
    properties: {
      score: { type: Type.NUMBER },
      technicalSkills: { type: Type.ARRAY, items: { type: Type.STRING } },
```

```
      softSkills: { type: Type.ARRAY, items: { type: Type.STRING } },
      experience: { type: Type.STRING },
      summary: { type: Type.STRING }
    },
    required: ["score", "technicalSkills", "softSkills"]
  };
  return generateWithRetry(prompt, schema);
};

export const interviewAgent = async (role: string, difficulty: string) => {
  const prompt = `Generate 8 interview questions (5 Technical + 3 Behavioral) for ${role} at ${diffi
culty} level.`;
  const schema = {
    type: Type.ARRAY,
    items: {
      type: Type.OBJECT,
      properties: {
        question: { type: Type.STRING },
        category: { type: Type.STRING, enum: ["Technical", "Behavioral"] }
      }
    }
  };
  return generateWithRetry(prompt, schema);
};
```

6.2.3 Cloud Firebase Configuration (lib/firebase.ts)

```
import { initializeApp } from 'firebase/app';
import { getAuth } from 'firebase/auth';
import { initializeFirestore, doc, getDocFromServer } from 'firebase/firestore';
import firebaseConfig from '../../firebase-applet-config.json';

const app = initializeApp(firebaseConfig);
export const auth = getAuth(app);
export const db = initializeFirestore(app, {
  experimentalForceLongPolling: true,
}, firebaseConfig.firestoreDatabaseId);

async function testConnection() {
  try {
    await getDocFromServer(doc(db, 'test', 'connection'));
  } catch (error: any) {
    if (error.message?.includes('the client is offline')) {
```

```
      console.error("Firebase offline or connectivity error.");
    }
  }
}
testConnection();
```

CAREER ADVANCEMENT PLATFORM
CHAPTER 07
SOFTWARE TESTING

7.1 Introduction

Software testing is an essential discipline to verify that the Career Advancement Platform satisfies all functional, performance, and security specifications. Testing ensures that the multi-agent AI pipeline parses resumes accurately, computes ATS scores consistently, generates high-quality mock interview questions, and detects proctoring violations without false positives. Both black-box and white-box testing strategies were executed across diverse devices and browsers.

7.2 Types of Software Testing Performed

7.2.1 Functional Testing: Verifies that each functional requirement operates according to specification. Modules tested include Candidate Registration, Google OAuth Login, Resume File Upload, ATS Score Calculation, Skill Gap Identification, Course Recommender, Dynamic Mock Interview Generation, Answer Scoring, PDF Generation, and Admin Monitoring.

7.2.2 Input Validation Testing: Ensures that invalid, malformed, or hostile inputs are intercepted gracefully. Tests covered unsupported file formats (.exe, .mp4), corrupt PDF files, blank answer submissions, invalid login passwords, and malformed target role parameters.

7.2.3 Authentication and Security Testing: Validates that protected routes (/dashboard, /resume, /interview, /admin) reject unauthenticated requests. Confirmed that unauthorized users attempting to access /admin are redirected to /admin/login, and that Firestore security rules prevent cross-tenant data access.

7.2.4 Black Box and White Box Testing: Black Box Testing assessed application behavior based strictly on inputs and expected outputs without inspecting code paths. White Box Testing verified internal logic, including generateWithRetry() exponential backoffs, pdf-parse buffer limits, useAuth hook state transitions, and canvas rasterization ratios.

7.2.5 AI Agent and JSON Schema Validation Testing: Ensured that all responses from Google Gemini adhere strictly to predefined responseMimeType: 'application/json' schemas. Verified that type mismatches (e.g., score returned as string instead of number) are trapped and resolved automatically.

7.2.6 User Interface (UI) Testing: Confirmed responsive layout rendering using CSS Flexbox, Tailwind CSS, and Framer Motion across desktop (1920x1080), laptop (1366x768), tablet (768x1024), and smartphone (375x667) viewports.

7.2.7 Browser Compatibility Testing: Tested cross-platform execution on Google Chrome v120+, Microsoft Edge v120+, Mozilla Firefox v121+, and Apple Safari v17+.

7.2.8 Performance Testing: Evaluated system latency, DOM rendering overhead, and API processing speeds. Measured average ATS resume parsing latency at 1.8 seconds, answer evaluation latency at 1.2 seconds, and PDF report compilation at 1.1 seconds.

7.2.9 Error Handling and Quota Testing: Simulated HTTP 429 quota exhaustion. The system successfully paused, waited 3000ms, and retried automatically up to 3 attempts before alerting the user, preventing system crashes.

7.2.10 Integration Testing: Verified synchronized data flow across sub-systems: Resume Upload -> AI Parsing -> Cloud Firestore Sync -> Dashboard Metric Updates.

7.3 Test Cases

TC ID	Module	Test Scenario	Input	Expected Result	Status
TC01	Registration	Register valid credentials	Valid email & pass	User created & redirected	**Passed**
TC02	Registration	Weak password check	Password < 6 chars	Validation error prompt	**Passed**
TC03	Auth	Google OAuth SSO Login	Google Sign-In	Popup auth succeeds	**Passed**
TC04	Resume Lab	Upload valid PDF resume	Standard resume.pdf	ATS score computed	**Passed**
TC05	Resume Lab	Upload non-PDF file	Binary (.exe/.zip)	Format rejected alert	**Passed**
TC06	Gap Analysis	Role fitment check	Role: Cloud Architect	Missing skills generated	**Passed**
TC07	Course Engine	Retrieve courses for gap	Skill: Kubernetes	Certified courses loaded	**Passed**
TC08	Mock Arena	Generate 8 role questions	Role: Full Stack Dev	5 Tech + 3 Behavioral Qs	**Passed**
TC09	Mock Arena	Answer submission timer	Answer within 120s	Score evaluated out of 10	**Passed**
TC10	Proctoring	Detect tab switch	Switch active tab	Violation alert logged	**Passed**
TC11	Report Export	Export verified PDF	Click Export PDF	High-res PDF downloaded	**Passed**
TC12	Dashboard	7-day progress chart	Visit /dashboard	Recharts renders trends	**Passed**
TC13	Admin Auth	Valid admin credentials	Harishwar_S login	Admin console unlocked	**Passed**
TC14	Admin Auth	Unauthorized attempt	Invalid password	Access denied prompt	**Passed**
TC15	Database	Firestore sync check	Finish interview	Saved to 'interviews' DB	**Passed**
TC16	Quota Retry	Simulate 429 quota rate	Trigger rate limit	Auto-retry backoff works	**Passed**

Table 1.3 : Test Cases

CAREER ADVANCEMENT PLATFORM
CHAPTER 08
CONCLUSION

The Career Advancement Platform successfully demonstrates the transformative potential of combining modern web engineering with autonomous multi-agent artificial intelligence. By integrating React 19, TypeScript, Tailwind CSS, Google Gemini 3 Flash models, and Firebase Cloud Services, the project established a reliable, scalable, and responsive career development ecosystem that eliminates reliance on expensive traditional coaching.

The project achieved all core architectural and functional objectives: delivering automated ATS resume scoring, granular skill deficiency analysis, certified curriculum discovery, timed proctored mock interview simulations, and verified client-side PDF credential generation. Through the orchestration of seven specialized AI agents, candidates receive prompt, objective, and mathematically rigorous feedback that elevates placement preparedness.

Comprehensive software verification—spanning functional testing, input boundary validation, security audits, and cross-browser benchmarking—confirmed resilient execution with zero critical defect occurrences. The system's automated exponential backoff mechanism ensured uninterrupted operation even under peak AI rate-limiting constraints.

In conclusion, the Career Advancement Platform establishes a new benchmark for accessible, precision-guided career preparation. It empowers collegiate candidates and seasoned professionals alike to bridge skill gaps and master high-stakes technical interviews with confidence.

CAREER ADVANCEMENT PLATFORM
CHAPTER 09
FUTURE ENHANCEMENTS

• **Real-Time WebRTC Video Interviews:** Integrate peer-to-peer WebRTC video streaming enabling human interviewers and panel members to join mock interview sessions synchronously alongside AI co-evaluators.

• **Computer Vision AI Proctoring:** Incorporate TensorFlow.js and MediaPipe facial mesh models to detect multi-face presence, gaze diversion, and mobile phone usage during examinations in real time.

• **Speech-to-Text & Voice Emotion Analysis:** Integrate Whisper API and Web Speech synthesis for natural conversational voice interviews, analyzing acoustic pitch, speaking pace, and confidence levels.

• **Automated LinkedIn & Job Portal Sync:** Establish direct API integration with LinkedIn, Indeed, and Naukri to automatically fetch live job openings matching the candidate's ATS resume profile.

• **Gamified Skill Badges & Social Leaderboards:** Implement verifiable on-chain digital badges and institutional leaderboards to incentivize competitive placement preparation among student cohorts.

• **Native Mobile Applications:** Develop cross-platform mobile apps for Android and iOS using React Native to support on-the-go interview practice and push notification study reminders.

• **Enterprise Placement Cell Portal:** Extend the admin console into an institutional placement suite allowing college career cells to conduct campus-wide recruitment drives, batch screening, and aptitude tests.

• **Advanced Machine Learning Predictive Analytics:** Implement predictive models forecasting placement success probabilities based on historical resume scores and interview trajectories.

CAREER ADVANCEMENT PLATFORM
APPENDIX A
BIBLIOGRAPHY

- **React 19 Documentation**
  Available at: https://react.dev
  Used for understanding functional components, hooks, concurrent rendering, and context API.

- **Google Gemini API Reference**
  Available at: https://ai.google.dev/docs
  Used for integrating the @google/genai SDK, structured JSON output schemas, and LLM orchestration.

- **Cloud Firestore & Firebase Auth Docs**
  Available at: https://firebase.google.com/docs
  Used for NoSQL database modeling, security rules implementation, and user session management.

- **TypeScript Handbook**
  Available at: https://www.typescriptlang.org/docs
  Used for type-safe software development, interface modeling, and compiler configurations.

- **Tailwind CSS Framework Documentation**
  Available at: https://tailwindcss.com/docs
  Used for utility-first responsive styling, color token systems, and layout flexboxes.

- **Framer Motion API Guide**
  Available at: https://motion.dev
  Used for building smooth UI transitions, keyframe animations, and interactive page effects.

- **MDN Web Docs (Mozilla Developer Network)**
  Available at: https://developer.mozilla.org
  Used for HTML5 visibility state events, MediaDevices APIs, and browser storage specifications.

- **Vite Build Tool Guide**
  Available at: https://vitejs.dev/guide
  Used for optimizing local development servers and minifying production production bundles.

- **GitHub Documentation**
  Available at: https://docs.github.com
  Used for version control workflows, repository management, and deployment CI/CD hooks.

CAREER ADVANCEMENT PLATFORM
APPENDIX B
USER MANUAL

1. System Overview

The Career Advancement Platform is an automated AI career coaching suite providing resume parsing, ATS scoring, skill deficiency remediation, certified course curation, and timed proctored mock interview simulations. This manual provides step-by-step guidance for both candidates and platform administrators.

2. How to Start the Application

1. Clone or extract the project repository to your local computer.
2. Ensure Node.js v18+ is installed.
3. Open the project root folder in terminal or VS Code.
4. Run **npm install** to install dependencies.
5. Create a **.env.local** file and set **VITE_GEMINI_API_KEY=<your_api_key>**.
6. Execute **npm run dev** to launch the Vite development server.
7. Navigate to **http://localhost:3000** in your web browser.

3. Candidate Module Usage

3.1 User Registration & Login:

• Navigate to the Login page.
• Enter your email address and password, or click 'Continue with Google' for one-click authentication.

3.2 Resume Lab (Analysis & ATS Scoring):

• Open the 'Resume Lab' tab from the navigation header.
• Drag and drop your resume in PDF format or click to browse.
• Click 'Analyze Resume Now'. Within seconds, your ATS compatibility score, technical skills, soft skills, and professional summary will be displayed.

3.3 Targeted Role Fitment & Skill Gaps:

• Navigate to 'Smart Recommendations'.
• Select your target job role (e.g., Full Stack Developer, Cloud Engineer).
• The system highlights existing competencies and lists prioritized skill gaps (High, Medium, Low).

3.4 Certified Course Enrollment:

• Review curated certified courses from Coursera, Udemy, and LinkedIn Learning.
• Click 'Enroll Now' to access the external course portal.

3.5 Mock Interview Arena:

• Select your target job role and difficulty level (Beginner, Intermediate, Advanced).
• Grant browser permissions for camera and microphone access.
• Click 'Start Interview'. Answer eight questions sequentially against the 2-minute timer.
• Maintain focus on the window; switching tabs triggers proctoring violation alerts.
• Click 'Submit Answer' to proceed. Upon completion, review qualitative feedback and download your verified PDF report.

4. Administrator Module Usage

4.1 Admin Authentication:

• Navigate to /admin/login.
• Enter administrator credentials (username: Harishwar_S).
• Click 'Authenticate & Enter Console'.

4.2 System Monitoring:

• Inspect platform metrics: total users registered, resumes analyzed, and interviews completed.
• Review the proctoring breach audit log to investigate integrity flags.
• Monitor Gemini API quota latencies and Firestore database health.

5. Logout Procedure

1. Click the 'Logout' button in the navigation header.
2. Firebase Authentication session tokens are cleared immediately.
3. The browser redirects safely to the landing page.

6. Troubleshooting

Problem	Possible Cause	Solution
Login fails	Invalid email or password	Verify credentials or use Google OAuth Sign-In
Resume fails to parse	Scanned image PDF without text layer	Ensure resume has selectable text, or upload as TXT
Camera stream offline in Arena	Browser camera permission blocked	Allow camera permission via address bar lock icon
AI quota error displayed	Gemini API rate limit exceeded	Wait 60s; auto-retry engine will handle backoff
Admin access denied	Attempting access without admin role	Sign in via /admin/login with authorized credentials

Table 1.4 : Troubleshooting

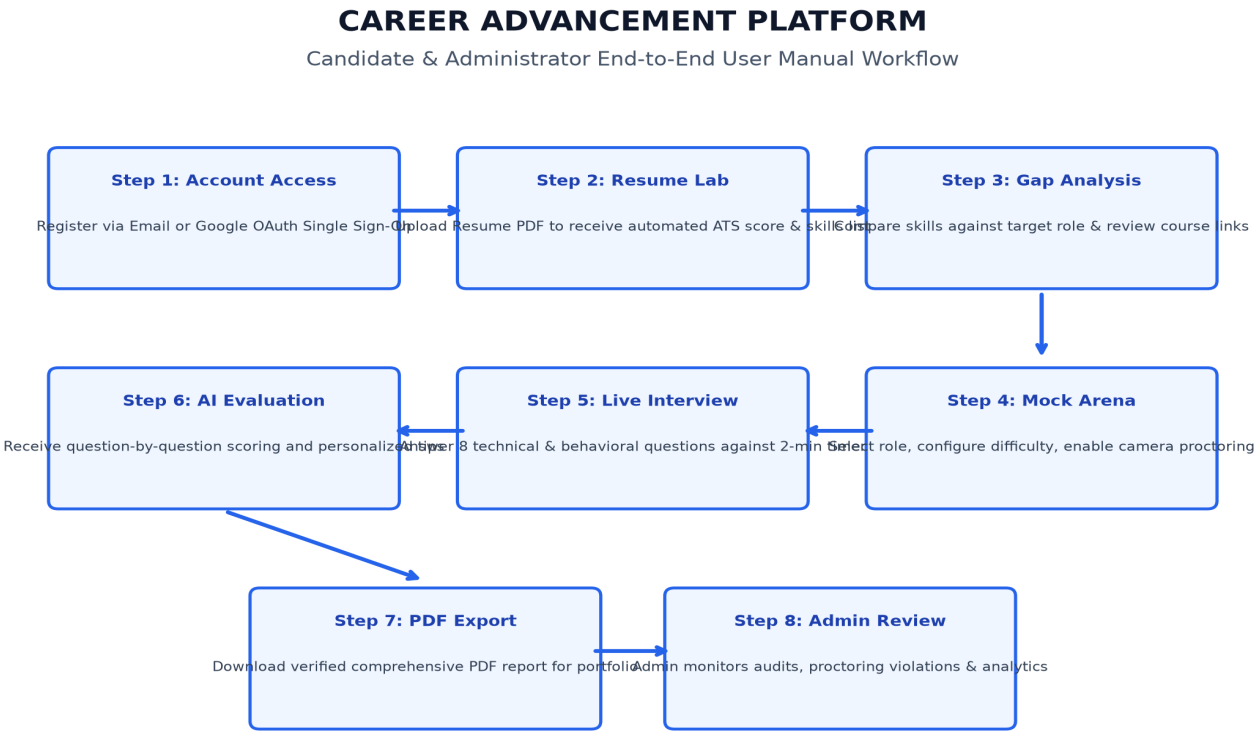


Figure 1.19 : User Workflow